Python Topic 13 — File I/O (Reading & Writing Files)

Full Stack Python + AI Roadmap • Taught in 3 layers: New to Python → Intermediate → Expert

🌱 Layer 1 — New to Python

File I/O means reading data **from** files and writing data **to** files on your computer.

```
# Write to a file
with open("notes.txt", "w") as file:
    file.write("Hello, Python!")

# Read from a file
with open("notes.txt", "r") as file:
    content = file.read()
    print(content)       # Hello, Python!
```

`open()` opens the file. `"w"` = "write mode," `"r"` = "read mode." The `with` block automatically closes the file when done (recall Topic 11).

👩‍💻 Layer 2 — Intermediate (real code + how it works)

File modes — the second argument to `open()`

Mode	Meaning
`"r"`	read (default) — errors if file doesn't exist
`"w"`	write — **creates or overwrites** (wipes existing content!)
`"a"`	append — adds to the end, keeps existing content
`"r+"`	read and write
`"x"`	create — errors if file already exists
`"rb"` / `"wb"`	binary mode (images, PDFs — recall `bytes`, Topic 2)

⚠️ **Note:** `"w"` **erases** the file's existing content. Use `"a"` to add without wiping.

Reading — several ways

```python
with open("data.txt", "r") as f:
    content = f.read()          # entire file as ONE string

with open("data.txt", "r") as f:
    lines = f.readlines()       # list of lines: ["line1\n", "line2\n"]

with open("data.txt", "r") as f:
    for line in f:              # iterate line by line (memory-efficient!)
        print(line.strip())     # strip() removes the trailing \n
```

The `for line in f:` pattern is best for big files — reads one line at a time instead of loading everything into memory.

Writing

```python
with open("output.txt", "w") as f:
    f.write("First line\n")     # \n = newline (you add it manually)
    f.write("Second line\n")

# write multiple lines at once
lines = ["apple\n", "banana\n", "cherry\n"]
with open("output.txt", "w") as f:
    f.writelines(lines)
```

Note: `write()` does **not** add a newline automatically — you include `\n` yourself.

🔑 Does the Second Write Overwrite the First?

No — both lines end up in the file, one after the other. The key: `"w"` wipes the file **once, at open time** — not on every `write()`.

```
After open("w"):     [                       ]  ← empty, cursor at start
After 1st write:     [First line\n           ]  ← cursor moved forward
After 2nd write:     [First line\nSecond line\n]  ← cursor moved again
```

Each `write()` writes at the cursor, then advances it — like a pen tip that keeps moving forward. The second write starts *where the first ended*, so writes stack up.

ℹ️ **What `"w"` actually wipes:** content from **previous times you opened the file** (earlier runs), erased the moment you re-open with `"w"`. Within a single `with` block, all writes accumulate.

Mode	On open	Cursor starts	Old content
`"w"`	wipes the file	position 0 (empty)	erased

`"a"`	keeps the file	at the end	preserved

⚠️ **The one exception:** if you manually `f.seek(0)` back to the start, the next write *will* overwrite. Without `seek()`, the cursor only moves forward.

Always use `with`

```
# ❌ manual — risky, might not close on error
f = open("data.txt")
data = f.read()
f.close()

# ✅ with — auto-closes
with open("data.txt") as f:
    data = f.read()
```

🧠 Layer 3 — Expert (internals & gotchas)

1. Working with JSON files (recall Topic 7)

The problem this solves: you have a dict in memory; you want to **save it to disk** so it survives after the program ends, then **read it back** later. (Like saving a document, then re-opening it.)

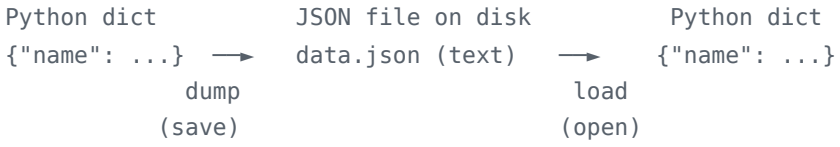
```
import json

# ---- WRITE: save a dict to a JSON file ----
data = {"name": "Akshay", "skills": ["React", "Python"]}
with open("data.json", "w") as f:
    json.dump(data, f, indent=2)
# json.dump(WHAT, WHERE) → converts dict to JSON text, writes it into file f
# indent=2 → pretty formatting

# ---- READ: load the file back into a dict ----
with open("data.json", "r") as f:
    data = json.load(f)
# json.load(f) → reads JSON text from file, converts back to a Python dict
```

After writing, `data.json` on disk contains:

```
{
  "name": "Akshay",
  "skills": [
    "React",
```

```
    "Python"
  ]
}
```

After reading, `data` is a real dict again — `data["name"]` → `"Akshay"`, `data["skills"][0]` → `"React"`.



ℹ️ **The `s` = "string":** `dump` / `load` work with **files**; `dumps` / `loads` work with **strings**. This example uses files (`data.json`), so no `s` .

Function	Works with	Use when
`json.dump(obj, file)`	a **file**	saving to a `.json` file
`json.dumps(obj)`	a **string**	need JSON as a string (e.g. API request)
`json.load(file)`	a **file**	reading from a `.json` file
`json.loads(str)`	a **string**	received a JSON string (e.g. API response)

2. `pathlib` — the modern way to handle paths

```python
from pathlib import Path

path = Path("data") / "files" / "notes.txt"   # OS-safe joining with /
path.exists()          # True/False
path.is_file()         # True/False
path.suffix            # ".txt"
path.stem              # "notes"
path.parent            # Path("data/files")
path.read_text()       # read whole file in one line!
path.write_text("hi")  # write in one line!

# list all .py files in a folder
for py_file in Path("src").glob("*.py"):
    print(py_file)
```

`pathlib` uses `/` to join paths (works on Windows *and* Mac/Linux). Cleaner than the older `os.path.join()` . Use it in all new code.

3. Encoding — always specify UTF-8 (production gotcha)

```
# ❌ relies on system default (varies by OS — bugs on Windows!)
with open("data.txt") as f:
    ...

# ✅ explicit encoding — consistent everywhere
with open("data.txt", encoding="utf-8") as f:
    ...
```

🚨 **Why:** without an explicit encoding, the same code behaves differently on Windows vs Mac/Linux, causing garbled text or crashes on emojis, accents, or non-English text. Always pass `encoding="utf-8"`.

4. CSV files

```
import csv

# Reading
with open("data.csv", "r", encoding="utf-8") as f:
    reader = csv.DictReader(f)          # each row becomes a dict
    for row in reader:
        print(row["name"], row["age"])

# Writing
with open("out.csv", "w", newline="", encoding="utf-8") as f:
    writer = csv.DictWriter(f, fieldnames=["name", "age"])
    writer.writeheader()
    writer.writerow({"name": "Akshay", "age": 26})
```

Note `newline=""` when writing CSV — it prevents blank rows on Windows. (For heavy data work you'll use `pandas`, but `csv` is built-in.)

5. The file cursor (position)

```
with open("data.txt") as f:
    f.read()        # reads everything → cursor now at END
    f.read()        # returns "" — nothing left to read!
    f.seek(0)       # move cursor back to START
    f.read()        # now works again
```

A file object tracks your position. After reading to the end, `seek(0)` to re-read. Surprises people when a second `read()` returns empty.

6. `os` for file operations & env vars

```python
import os
os.path.exists("data.txt")     # check existence (or use pathlib)
os.remove("data.txt")          # delete a file
os.rename("old.txt", "new.txt")
os.makedirs("a/b/c", exist_ok=True)   # create nested folders

# environment variables (crucial for API keys!)
os.environ.get("OPENAI_API_KEY")      # read env var safely
```

✅ **For your AI work:** reading env vars is how you handle API keys and secrets (never hardcode them). Usually paired with `python-dotenv` to load a `.env` file.

🎯 Interview Q&A Cards

Q: Does a second `write()` overwrite the first?

A: No. `"w"` wipes the file once at open; each `write()` then appends at the cursor and advances it, so writes stack. Only `seek()` moves the cursor back to overwrite.

Q: `json.dump` vs `json.dumps`?

A: `dump` / `load` work with **files**; `dumps` / `loads` (the `s` = string) work with **strings**. Use file versions for `.json` files, string versions for API payloads.

Q: Why prefer `with open(...)`?

A: It guarantees the file closes even if an exception occurs, avoiding resource leaks and data-loss from unflushed buffers.

Q: Why always pass `encoding="utf-8"`?

A: The default encoding varies by OS, so the same code can garble or crash on non-ASCII text across platforms. Explicit UTF-8 makes behavior consistent.

🪝 **Memory hook:** *`with open` auto-closes. `w` wipes once at open (writes then move forward), `a` adds. `dump` / `load` = files, `dumps` / `loads` = strings. Always `encoding="utf-8"`.*

✅ **One-liner for interviews:** "Always use `with open(...)` so files auto-close. Know the modes — `w` overwrites, `a` appends. Use `json.dump` / `load` for files vs `dumps` / `loads` for strings. Prefer `pathlib` over `os.path`, and always specify `encoding='utf-8'`."